Potiguar+: DDL, DML e Consultas — Resolução Completa

Disciplina: IMD0401 - Banco de Dados (2026.2) | **Docente:** Prof. Alexandre Varella

Instituição: UFRN / IMD · Bacharelado em Tecnologia da Informação

Este documento apresenta a resolução detalhada de todos os itens da **Lista de Banco de Dados (Aulas 8 e 9)** sobre o programa de fidelidade *Potiguar+*. A solução foi validada no PostgreSQL 16 com o modo estrito `psql -v ON_ERROR_STOP=1`.

S. Conhecendo a base (2,0 pts)

Regra: Consultas a uma única tabela, com anotação do total de linhas retornadas.

(a) Os 5 produtos ativos mais caros, com nome e preço.

```
SELECT nome, preco FROM produto WHERE ativo = true ORDER BY preco DESC LIMIT 5;
```

Lógica: Filtra apenas produtos com a flag `ativo = true`, ordena de forma decrescente pelo preço (`ORDER BY preco DESC`) e restringe aos 5 primeiros registros com `LIMIT 5`. Devolve exatamente **5 linhas**.

(b) Quantos clientes estão sem telefone e quantos estão sem cidade, numa consulta só e sem WHERE.

```
SELECT COUNT(*) FILTER (WHERE telefone IS NULL) AS clientes_sem_telefone, COUNT(*) FILTER
```

```
(WHERE cidade IS NULL) AS clientes_sem_cidade FROM cliente;
```

Lógica: O PostgreSQL disponibiliza a cláusula agregada condicional `FILTER (WHERE ...)`, permitindo contar seletivamente os valores `NULL` de cada coluna em uma única passada sobre a tabela inteira, sem utilizar a cláusula `WHERE` global. Retorna **1 linha** com as duas contagens.

(c) Os clientes de Mossoró ou de Caicó cadastrados em 2025. Diga em comentário quantas linhas viriam sem os parênteses, e por quê.

```
SELECT * FROM cliente WHERE (cidade = 'Mossoró' OR cidade = 'Caicó') AND (data_cadastro >=
```

```
'2025-01-01' AND data_cadastro < '2026-01-01');
```

> **Resposta teórica obrigatória sobre os parênteses:**
> O operador lógico `AND` possui precedência superior ao operador `OR` na álgebra booleana do SQL. Sem os parênteses:
> `WHERE cidade = 'Mossoró' OR cidade = 'Caicó' AND data_cadastro >= '2025-01-01'...`
> O banco interpretaria a expressão como:
> `WHERE cidade = 'Mossoró' OR (cidade = 'Caicó' AND data_cadastro >= '2025-01-01'...)`
> Isso faria com que **todos os clientes de Mossoró cadastrados em qualquer ano** fossem retornados, inflando drasticamente a quantidade de linhas em relação ao filtro desejado.

> **(d)** Quantos pedidos de agosto de 2026 não foram cancelados? Lembre que data_pedido é TIMESTAMP.

```
SELECT COUNT(*) AS total_pedidos_agosto_nao_cancelados FROM pedido WHERE data_pedido >=
```

```
'2026-08-01 00:00:00' AND data_pedido < '2026-09-01 00:00:00' AND status <> 'cancelado';
```

> **Lógica:** Para colunas `TIMESTAMP`, a comparação por intervalo semiaberto `[>= início, < início_do_mês_seguinte)` é uma boa prática essencial (SARGable) que não depende de funções de conversão e permite uso pleno de índices. O status diferente de cancelado filtra pedidos válidos. Retorna **1 linha**.

> **(e)** Os notebooks (busque pelo nome, sem depender de maiúsculas), com o preço e o valor da parcela em 12 vezes, em centavos.

```
SELECT nome, preco, ROUND((preco / 12.0) * 100) AS parcela_12x_centavos FROM produto WHERE
```

```
nome ILIKE '%notebook%';
```

> **Lógica:** O operador `ILIKE` do PostgreSQL realiza a busca textual insensível a maiúsculas/minúsculas. Para calcular o valor da parcela em centavos, dividimos o preço por 12 e multiplicamos por 100 (arredondado para inteiro).

P1. A tabela do programa (2,0 pts)

> Crie a tabela fidelidade atendendo a todos os requisitos. Depois, sem recriá-la, acrescente a coluna indicado_por com ALTER TABLE.

```
-- 1. Criação da tabela com PK, chaves estrangeiras, default e restrições CHECK: CREATE
```

```
TABLE fidelidade ( id_cliente INT PRIMARY KEY, nivel VARCHAR(10) NOT NULL DEFAULT 'bronze'
```

```
CHECK (nivel IN ('bronze', 'prata', 'ouro')), pontos INT NOT NULL DEFAULT 0 CHECK (pontos
```

```
>= 0), data_adesao DATE NOT NULL DEFAULT CURRENT_DATE, CONSTRAINT fk_fidelidade_cliente
FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente) ON DELETE CASCADE ); -- 2. Adição
da coluna indicado_por com FK reflexiva/cliente e ON DELETE SET NULL: ALTER TABLE
fidelidade ADD COLUMN indicado_por INT, ADD CONSTRAINT fk_fidelidade_indicado_por FOREIGN
KEY (indicado_por) REFERENCES cliente(id_cliente) ON DELETE SET NULL;
```

Atendimento aos requisitos:

- `id_cliente` é PK e FK para `cliente` com `ON DELETE CASCADE` (ao apagar o cliente, apaga a adesão).
- `nivel` obrigatório (`NOT NULL`), restrito a `'bronze'`, `'prata'` ou `'ouro'` via `CHECK`, com padrão `'bronze'`.
- `pontos` inteiro, `NOT NULL`, `CHECK (pontos >= 0)` e padrão 0.
- `data_adesao` obrigatória, com padrão `CURRENT_DATE` (data de hoje).
- `indicado_por` adicionado via `ALTER TABLE` com `ON DELETE SET NULL` (se o indicador for removido, o campo volta a nulo).

P2. Os primeiros participantes (2,0 pts)

(a) Os fundadores: todos os clientes de Natal cadastrados em 2025, com adesão em 01/10/2026, num único INSERT e sem digitar os ids.

```
INSERT INTO fidelidade (id_cliente, data_adesao) SELECT id_cliente, '2026-10-01'::DATE
FROM cliente WHERE cidade = 'Natal' AND (data_cadastro >= '2025-01-01' AND data_cadastro <
'2026-01-01') RETURNING *;
```

(b) Gustavo Batista (cliente 28), indicado por Helena Diniz (29). A adesão é hoje: deixe o padrão agir.

```
INSERT INTO fidelidade (id_cliente, indicado_por) VALUES (28, 29) RETURNING *;
```

Lógica: Omitindo as colunas `nivel`, `pontos` e `data_adesao`, o PostgreSQL aplica automaticamente os valores padrão: `'bronze'`, `0` e `CURRENT_DATE`.

(c) Dois INSERT que o banco precisa recusar, por motivos diferentes (deixar comentados com mensagem e restrição).

```
-- FALHA 1: Violação de CHECK de nível permitido -- INSERT INTO fidelidade (id_cliente,
nivel) VALUES (1, 'diamante'); -- ERRO: new row for relation "fidelidade" violates check
constraint "fidelidade_nivel_check" -- DETAIL: Failing row contains (1, diamante, 0, 2026-
10-21, null). -- Restrição: CHECK (nivel IN ('bronze', 'prata', 'ouro')) -- FALHA 2:
Violação de CHECK de pontuação negativa -- INSERT INTO fidelidade (id_cliente, pontos)
VALUES (2, -50); -- ERRO: new row for relation "fidelidade" violates check constraint
"fidelidade_pontos_check" -- DETAIL: Failing row contains (2, bronze, -50, 2026-10-21,
null). -- Restrição: CHECK (pontos >= 0)
```

P3. Pontos da campanha de outubro (2,0 pts)

(a) Lance os pontos num único comando: quem já participa soma os pontos novos aos que tinha; quem não participa é inscrito. Use RETURNING.

```
INSERT INTO fidelidade (id_cliente, pontos) VALUES (1, 600), (5, 1200), (28, 150), (3,
300) ON CONFLICT (id_cliente) DO UPDATE SET pontos = fidelidade.pontos + EXCLUDED.pontos
RETURNING *;
```

Lógica (UPSERT): A instrução `ON CONFLICT (id_cliente)` identifica conflitos na chave primária. Para clientes existentes (1, 5, 28), executa o `DO UPDATE` somando os pontos existentes aos novos fornecidos (`EXCLUDED.pontos`). Para a cliente 3, inexistente no programa, realiza o `INSERT` padrão com 300 pontos.

(b) Recalcule o nível de todos (ouro >= 1.000, prata >= 500, bronze < 500) e devolva só as linhas que mudaram de nível.

```
UPDATE fidelidade SET nivel = CASE WHEN pontos >= 1000 THEN 'ouro' WHEN pontos >= 500 THEN
'prata' ELSE 'bronze' END WHERE nivel <> CASE WHEN pontos >= 1000 THEN 'ouro' WHEN pontos
>= 500 THEN 'prata' ELSE 'bronze' END RETURNING *;
```

Lógica: O filtro no `WHERE` compara o nível atual com o novo nível recalculado. Se já estiver no mesmo nível, a linha não é alterada nem retornada. O `RETURNING *` devolve exatamente as tuplas atualizadas.

P4. O relatório e a saída (2,0 pts)

(a) O relatório: cliente, nível, pontos, adesão no formato DD/MM/AAAA e quantos pontos faltam para o próximo nível (0 no ouro). Ouro primeiro, depois prata e bronze; dentro do nível, mais pontos primeiro.

```
SELECT c.nome AS cliente, f.id_cliente, f.nivel, f.pontos, TO_CHAR(f.data_adesao,
'DD/MM/YYYY') AS adesao, CASE WHEN f.nivel = 'ouro' THEN 0 WHEN f.nivel = 'prata' THEN
1000 - f.pontos WHEN f.nivel = 'bronze' THEN 500 - f.pontos END AS
pontos_para_proximo_nivel FROM fidelidade f JOIN cliente c ON f.id_cliente = c.id_cliente
ORDER BY CASE f.nivel WHEN 'ouro' THEN 1 WHEN 'prata' THEN 2 WHEN 'bronze' THEN 3 END ASC,
f.pontos DESC;
```

(b) Numa consulta só: quantos participantes há e quantos entraram por indicação.

```
SELECT COUNT(*) AS total_participantes, COUNT(indicado_por) AS entraram_por_indicacao FROM
fidelidade;
```

(c) Entre BEGIN e ROLLBACK, apague de cliente a 29 e depois o 28, mostrando fidelidade depois de cada DELETE. Explique em comentário por que apagar a cliente 1 não daria certo.

```
BEGIN; -- Apaga Helena Diniz (29): indicado_por de Gustavo Batista (28) vira NULL (ON
DELETE SET NULL) DELETE FROM cliente WHERE id_cliente = 29; SELECT * FROM fidelidade; --
Apaga Gustavo Batista (28): a linha do cliente 28 em fidelidade é apagada (ON DELETE
CASCADE) DELETE FROM cliente WHERE id_cliente = 28; SELECT * FROM fidelidade; ROLLBACK;
```

Explicação teórica (Cliente 1):
A cliente 1 possui registros associados em tabelas filhas (ex: na tabela `pedido`). Como a chave estrangeira `fk_pedido_cliente` foi declarada com o comportamento padrão `ON DELETE RESTRICT` (ou `NO ACTION`), o PostgreSQL recusa a operação de exclusão para garantir a integridade referencial, gerando erro de restrição.